# Manipulação e Organização de Arquivos de Dados

Algoritmos e Estruturas de Dados II

Professor: Rafael Alexandre




Fabian Giovanna Marcolino dos Santos

Matrícula: 24.2.8130

## 1. Objetivo e visão geral

O programa simula a persistência de registros de alunos em um arquivo binário alunos.dat dividido em **blocos de tamanho fixo** (informado pelo usuário), comparando três estratégias de organização: (1) registros de **tamanho fixo**; (2) registros de **tamanho variável contíguos** (sem espalhamento) e (3) registros de **tamanho variável espalhados** (fragmentados entre blocos). Após a gravação, o programa relê o arquivo, confere que os registros recuperados são idênticos aos originais e exibe estatísticas de uso dos blocos e um mapa de ocupação.

Implementação em **Python 3**, apenas com a biblioteca padrão. Código organizado em módulos: registro.py (campos e serialização), gerador.py (dados fictícios), armazenamento.py (blocos, gravação e leitura), estatisticas.py (métricas e mapa), main.py (interface) e comparar.py (experimentos).

## 2. Dados utilizados

Os registros são gerados automaticamente por gerador.py a partir de uma semente (42), de modo que os experimentos sejam reprodutíveis e as três estratégias sejam comparadas sobre exatamente os mesmos 1000 registros. Os nomes combinam prenomes e sobrenomes comuns (com tamanhos variados, até dois prenomes e sobrenomes compostos); pai e mãe compartilham sobrenomes com o aluno. Todo texto é ASCII (sem acentos), de modo que 1 caractere = 1 byte.

| Campo | Largura máx. (bytes) | Observação |
|---|---|---|
| matrícula | 9 | dígitos ASCII; único |
| nome | 50 | até 50 caracteres |
| CPF | 11 | dígitos, com verificadores válidos |
| curso | 30 | |
| filiação (mãe) | 30 | |
| filiação (pai) | 30 | |
| ano de ingresso | 4 | 2008 a 2025 |
| CA | 5 | texto d.dd, 0,00 a 10,00 |
| Total (registro fixo) | 169 | maior registro possível |

## 3. Decisões de projeto

### 3.1 Registro de tamanho fixo

Cada campo ocupa sua largura máxima, completada com o caractere #; o registro tem sempre **169 bytes** (soma das larguras). Exemplo (registro 1):

```
201103278Fernanda                                                
```

### 3.3 Tratamento do limite do bloco

- **Contíguo:** se ocupado + 3 + tamanho > B, o registro inteiro é movido para o próximo bloco; o espaço restante no bloco atual é desperdiçado. Se um único registro (com cabeçalho) não couber nem em um bloco vazio, o programa recusa o valor de *B* e informa o mínimo necessário.
- **Espalhado:** o registro é gravado até o último byte livre do bloco; o restante prossegue no bloco seguinte, com um novo cabeçalho marcado como continuação. Se sobrarem menos de 4 bytes (nem cabeçalho + 1 byte de dado), abre-se um novo bloco, pois um fragmento sem dados seria inútil. Funciona para qualquer $B \geq 4$, inclusive menor que um registro (o registro é distribuído em vários blocos).
- **Todos os modos:** cada bloco é gravado com exatamente *B* bytes; o espaço não usado é completado com 0x00 (o byte de *flags* 0 sinaliza ao leitor o fim dos dados do bloco). O tamanho do arquivo é sempre nº de blocos × B.

### 3.4 Pseudocódigo do espalhamento

```
para cada registro R:
  corpo ← campos de R separados por '|'; pos ← 0; primeiro ← verdadeiro
  enquanto pos < |corpo|:
    livre ← B − ocupado(bloco atual)
    se livre < 4: abrir novo bloco; continuar
    restante ← |corpo| − pos
    se 3 + restante ≤ livre: n ← restante; flags ← 0x01            (cabe tudo)
    senão:                   n ← livre − 3;  flags ← 0x01 | 0x02   (continua)
    se não primeiro: flags ← flags | 0x04                          (continuação)
    gravar cabeçalho(flags, n) + corpo[pos : pos+n] no bloco atual
    pos ← pos + n; primeiro ← falso
```

## 4. Estatísticas calculadas

- **Total de blocos** utilizados (= tamanho do arquivo / *B*).
- **Ocupação média:** média, sobre todos os blocos, de bytes gravados no bloco / B (inclui cabeçalhos e preenchimento #, que são bytes de fato ocupados no bloco).
- **Blocos parcialmente utilizados:** blocos com ocupação < 100%; também é informado o número de blocos 100% cheios.
- **Eficiência de armazenamento:** bytes de dados úteis / (nº de blocos × B). “Dados úteis” são apenas os bytes de conteúdo dos campos — excluem-se preenchimento #, separadores |, cabeçalhos e a sobra no fim dos blocos. Assim a eficiência distingue

ocupação “de verdade” de ocupação por *overhead*: o formato fixo pode ter ocupação alta e eficiência baixa.

- **Mapa de ocupação:** uma linha por bloco (bytes, percentual, barra em texto, nº de registros e marcas de fragmento). O gráfico de barras opcional está na seção 5.

O exemplo do enunciado (blocos com 512, 486 e 210 bytes) produz 1208/1536 = 78,6% de ocupação; esse caso está coberto por um teste automatizado da função de estatísticas.

## 5. Resultados

### 5.1 Cenário principal: 1000 registros, bloco de 512 bytes

| Estratégia | Blocos | Ocupação média | Blocos cheios | Blocos parciais | Bytes gravados | Capacidade (B) | Eficiência |
|---|---|---|---|---|---|---|---|
| Tamanho fixo | 334 | 98,8% | 0 | 334 | 169000 | 171008 | 61,5% |
| Variável contíguo | 251 | 89,7% | 0 | 251 | 115214 | 128512 | 81,9% |
| Variável espalhado | 227 | 99,7% | 221 | 6 | 115871 | 116224 | 90,5% |

Os arquivos gerados (entregues em dados/) têm exatamente blocos × 512 bytes: alunos_fixo.dat (171008 B), alunos_variavel_contiguo.dat (128512 B) e alunos_variavel_espalhado.dat (116224 B). Cada um acompanha um _mapa.txt com o mapa completo de todos os blocos. Trecho do mapa (início de cada estratégia e último bloco do espalhado):

```
Bloco   1:   507 bytes ( 99,0% cheio) |############################.| 3 reg.
Bloco   2:   507 bytes ( 99,0% cheio) |############################.| 3 reg.
Bloco   3:   507 bytes ( 99,0% cheio) |############################.| 3 reg.
Bloco   1:   430 bytes ( 84,0% cheio) |########################.....| 4 reg.
Bloco   2:   450 bytes ( 87,9% cheio) |########################....| 4 reg.
Bloco   3:   484 bytes ( 94,5% cheio) |############################..| 4 reg.
Bloco   4:   426 bytes ( 83,2% cheio) |########################.....| 4 reg.
Bloco   1:   512 bytes (100,0% cheio) |#############################| 5 reg. [fragmento ->]
Bloco   2:   512 bytes (100,0% cheio) |#############################| 5 reg. [<- continuacao] [fragmento ->]
Bloco   3:   512 bytes (100,0% cheio) |#############################| 4 reg. [<- continuacao] [fragmento ->]
Bloco 227:   170 bytes ( 33,2% cheio) |#########.....................| 1 reg. [<- continuacao]
```

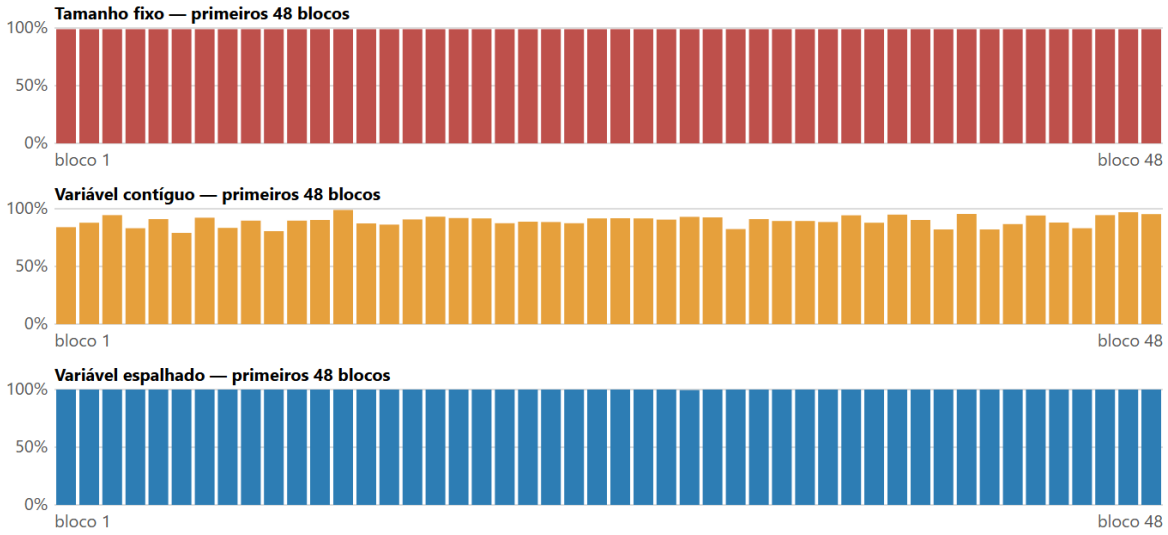



Figura 1 — Ocupação (%) dos 48 primeiros blocos em cada estratégia (B = 512).

## 5.2 Efeito do tamanho do bloco

| Bloco (B) | Estratégia | Blocos | Ocup. média | Cheios | Parciais | Arquivo (B) | Eficiência |
|---|---|---|---|---|---|---|---|
| 256 | Tamanho fixo | 1000 | 66,0% | 0 | 1000 | 256000 | 41,1% |
| 256 | Variável contíguo | 526 | 85,6% | 3 | 523 | 134656 | 78,1% |
| 256 | Variável espalhado | 456 | 99,8% | 447 | 9 | 116736 | 90,1% |
| 512 | Tamanho fixo | 334 | 98,8% | 0 | 334 | 171008 | 61,5% |
| 512 | Variável contíguo | 251 | 89,7% | 0 | 251 | 128512 | 81,9% |
| 512 | Variável espalhado | 227 | 99,7% | 221 | 6 | 116224 | 90,5% |
| 1024 | Tamanho fixo | 167 | 98,8% | 0 | 167 | 171008 | 61,5% |
| 1024 | Variável contíguo | 120 | 93,8% | 0 | 120 | 122880 | 85,6% |
| 1024 | Variável espalhado | 113 | 99,8% | 109 | 4 | 115712 | 90,9% |
| 2048 | Tamanho fixo | 84 | 98,2% | 0 | 84 | 172032 | 61,2% |
| 2048 | Variável contíguo | 59 | 95,3% | 0 | 59 | 120832 | 87,1% |
| 2048 | Variável espalhado | 57 | 98,8% | 54 | 3 | 116736 | 90,1% |
| 4096 | Tamanho fixo | 42 | 98,2% | 0 | 42 | 172032 | 61,2% |
| 4096 | Variável contíguo | 29 | 97,0% | 0 | 29 | 118784 | 88,6% |
| 4096 | Variável espalhado | 29 | 97,1% | 28 | 1 | 118784 | 88,6% |

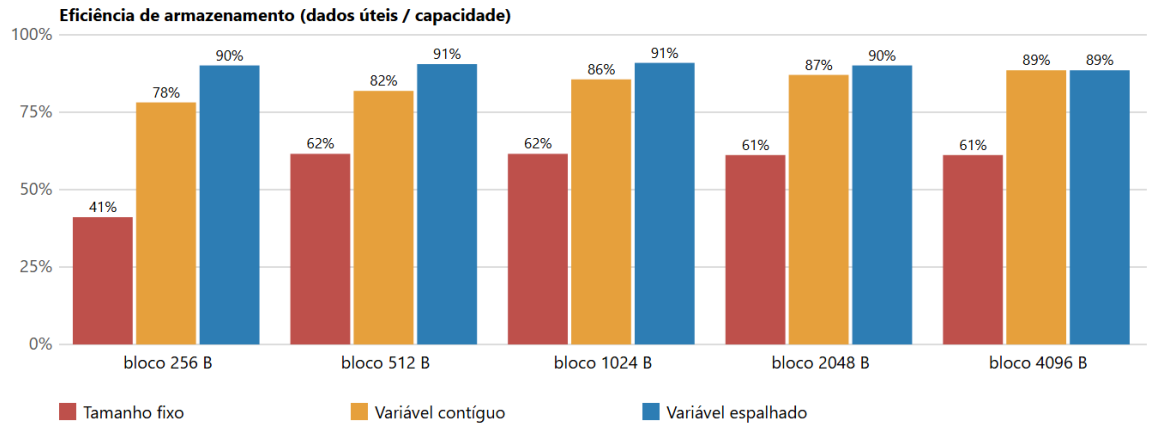



Figura 2 — Eficiência de armazenamento por tamanho de bloco.

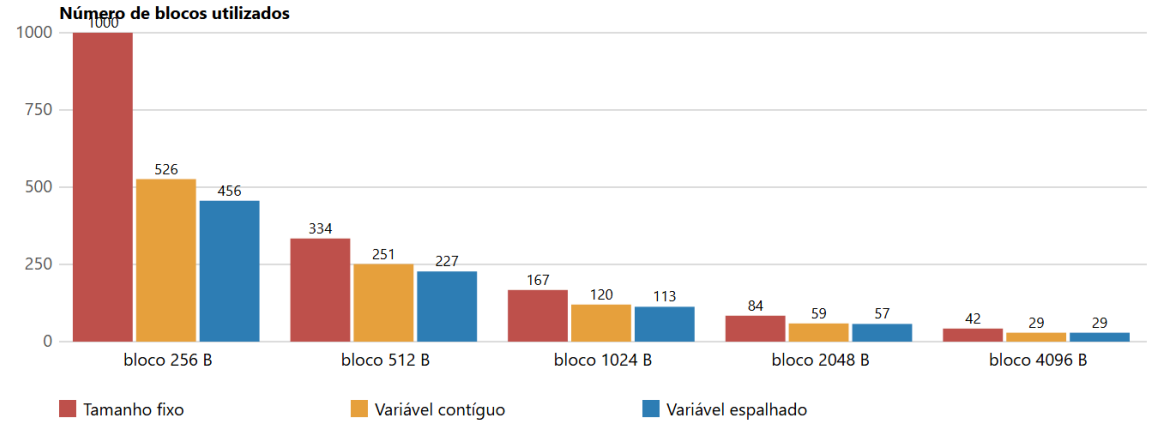



Figura 3 — Número de blocos utilizados por tamanho de bloco (1000 registros).

## 6. Análise

- **Tamanho fixo.** Com B = 512 cabem 3 registros de 169 bytes (sobram 5 bytes por bloco), logo a *ocupação* é alta (98,8%), mas a *eficiência* é a menor (61,5%): em média só 105,2 dos 169 bytes de cada registro são dados; o resto é preenchimento. O desempenho depende de B mod 169: com B = 256 cabe 1 registro por bloco (apenas 66,0% de ocupação, 1000 blocos), e B = 512 e B = 1024 têm a mesma eficiência porque ambos guardam ≈ 3 registros a cada 512 bytes. Em compensação é a organização mais simples: o endereço do *n*-ésimo registro é calculado diretamente e nenhum registro cruza blocos.
- **Variável contíguo.** Elimina o preenchimento e usa 251 blocos (25% menos que o fixo), mas perde espaço no fim de cada bloco, quando o próximo registro não cabe: ocupação média de 89,7% e nenhum bloco 100% cheio. Esse desperdício diminui quando B

cresce em relação ao tamanho do registro (eficiência de 78,1% com B = 256 contra 88,6% com B = 4096).

- **Variável espalhado.** É a mais compacta (227 blocos; 221 deles 100% cheios; apenas 6 parciais) e a de maior eficiência (90,5%). O custo é um cabeçalho extra de 3 bytes por fragmento e a necessidade de ler dois blocos para recuperar um registro fragmentado (aqui, quase todos os blocos terminam em fragmento), além de lógica de leitura mais complexa. Com B grande a vantagem sobre o contíguo diminui (em B = 4096 ambos usam o mesmo nº de blocos).
- **Conclusão.** Há um *trade-off* entre espaço e simplicidade/desempenho de acesso: fixo (simples, desperdiça espaço) → variável contíguo (compacta, perde o fim de cada bloco, sem acessos duplos) → variável espalhado (aproveita quase todo o espaço, mas acessos duplos e mais cabeçalhos). Em sistemas reais o espalhamento é usado principalmente quando os registros são grandes em relação ao bloco.

## 7. Validação

- **Releitura do arquivo:** após cada gravação, o programa lê o .dat bloco a bloco, reconstrói os registros (juntando fragmentos) e compara com os originais; o resultado é impresso ("Verificação: OK"). O tamanho do arquivo também é conferido (= blocos × B).
- **Testes unitários** (tests/test_armazenamento.py, 17 testes): ida e volta em todos os modos e vários valores de B (inclusive B = 4, o caso extremo do espalhado); nenhum bloco excede B; fixo nunca cruza bloco; contíguo nunca fragmenta; espalhado usa ≤ blocos que o contíguo; erros para blocos pequenos; exemplo numérico do enunciado (78,6%); bytes úteis idênticos nas três estratégias.

## 8. Como executar

```
cd src
python main.py                          # modo interativo (pergunta tudo)
python main.py --registros 1000 --bloco 512 --modo fixo --saida ../dados/alunos_fixo.dat
python main.py --registros 1000 --bloco 512 --modo variavel --espalhamento nao --saida ../dados/alunos_variavel_contiguo.dat
python main.py --registros 1000 --bloco 512 --modo variavel --espalhamento sim --saida ../dados/alunos_variavel_espalhado.dat
python main.py ... --mapa completo              # imprime o mapa de todos os blocos
python comparar.py                      # tabela comparativa (gera ../resultados/comparacao.csv)
cd .. && python -m unittest discover -s tests -v     # testes
```

Sem parâmetros, o programa pergunta: número de registros, modo (fixo/variável), espalhamento (se variável) e tamanho do bloco.